

INSTITUT TEKNOLOGI DEL

OOP CONCEPT
CLASS & OBJECT

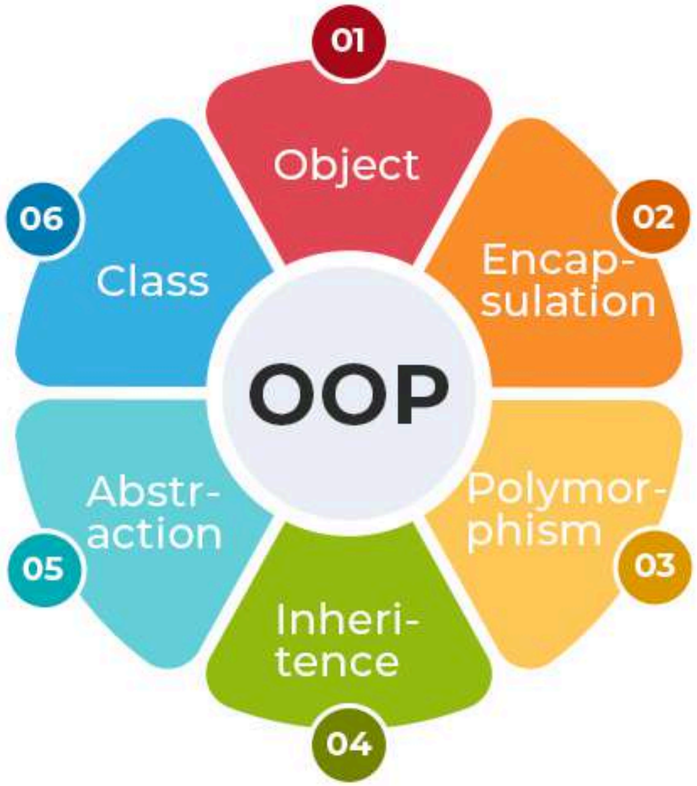
Fakultas Vokasi | D4TRPL | Institut Teknologi Del

Week 3 Sesi 1 - CDN - Tahun Ajaran 2026/2027

Konten

- **Objects and Classes**
  - Objects, Classes, and Instances
  - Fundamentals of Objects
- **Constructors and Object Initialization**
  - Initializing Instance Variables
  - Constructors
  - Garbage Collection

Object Oriented in Java



Java is an Object-Oriented Language. As a language that has the Object Oriented feature, Java supports the following fundamental concepts:

- **Classes & Objects**
  - Variable
  - Method
  - Message Parsing
- Inheritance
- Encapsulation
- Polymorphism
- Abstraction

Objects and Classes

In the broadest term, an **object is a thing**, both **tangible and intangible**, that we can imagine.

- For a program to keep track of student residents of a college dormitory, we may have many **Student**, **Room**, and **Floor** as **objects**.
- For another program to keep track of customers and inventory for a bicycle shop, **we may have?**

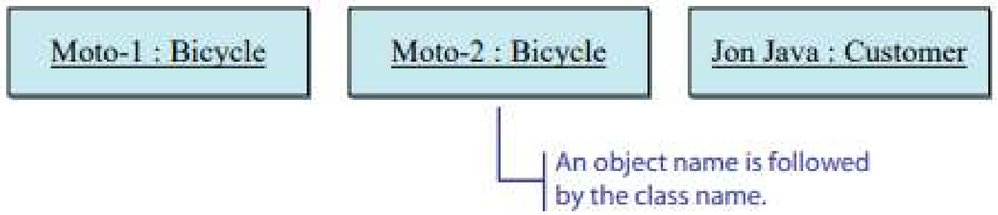


Figure 1.2 Two Bicycle objects with the names Moto-1 and Moto-2 and one Customer object with the name Jon Java.

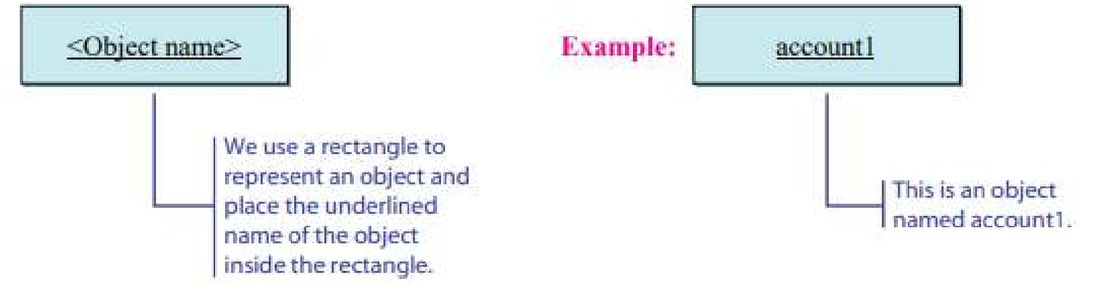


Figure 1.1 A graphical representation of an object.

An object is comprised of data and operations that manipulate these data.

- For example, **a Student** object may consist of **data** such as name, gender, birth date, home address, phone number, and age and **operations** for assigning and changing these data values.

Objects and Classes

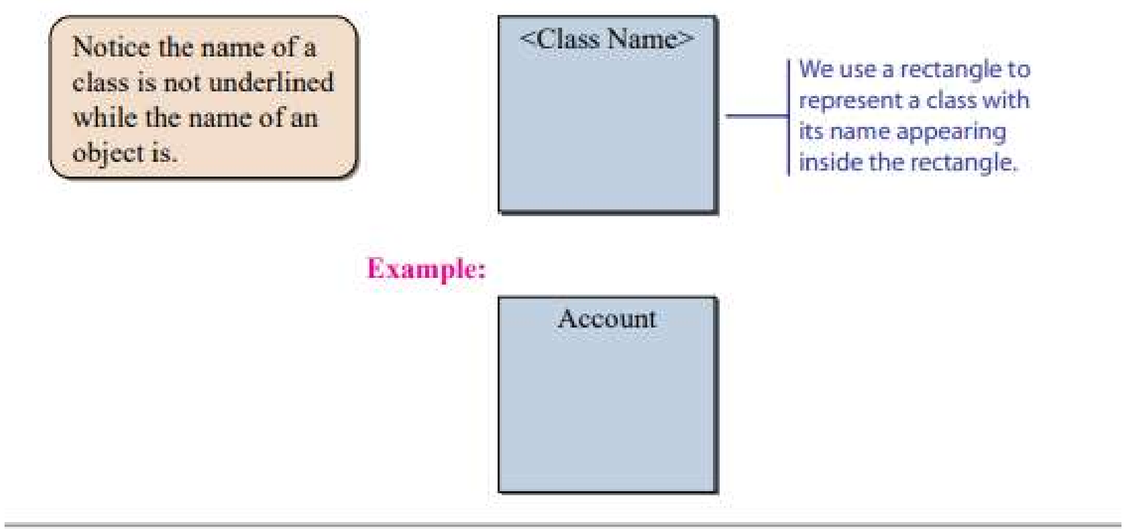


Figure 1.3 A graphical representation of a class.

For the computer **to be able to create an object**, we must provide a definition, called a **class.**

A class is a kind of mold or **template that dictates what objects** can and cannot do. An object is called an instance of a class.

- An object is called an instance of a class.
- An object is an instance of exactly one class.
- An instance of a class belongs to the class.

Once a class is defined, we can create as many instances of the class as a program requires.

In general, properties of java classes can include these characteristics in order:

- Class is not a real-world entity. It is just a **template** or **blueprint**, or a prototype from which objects are created.
- A class itself does not occupy memory for its attributes and methods until an object is instantiated.
- A Class in Java can **contain data** members, **methods**, a **constructor**, **nested classes**, and **interfaces**.

Class and Instance Data Values

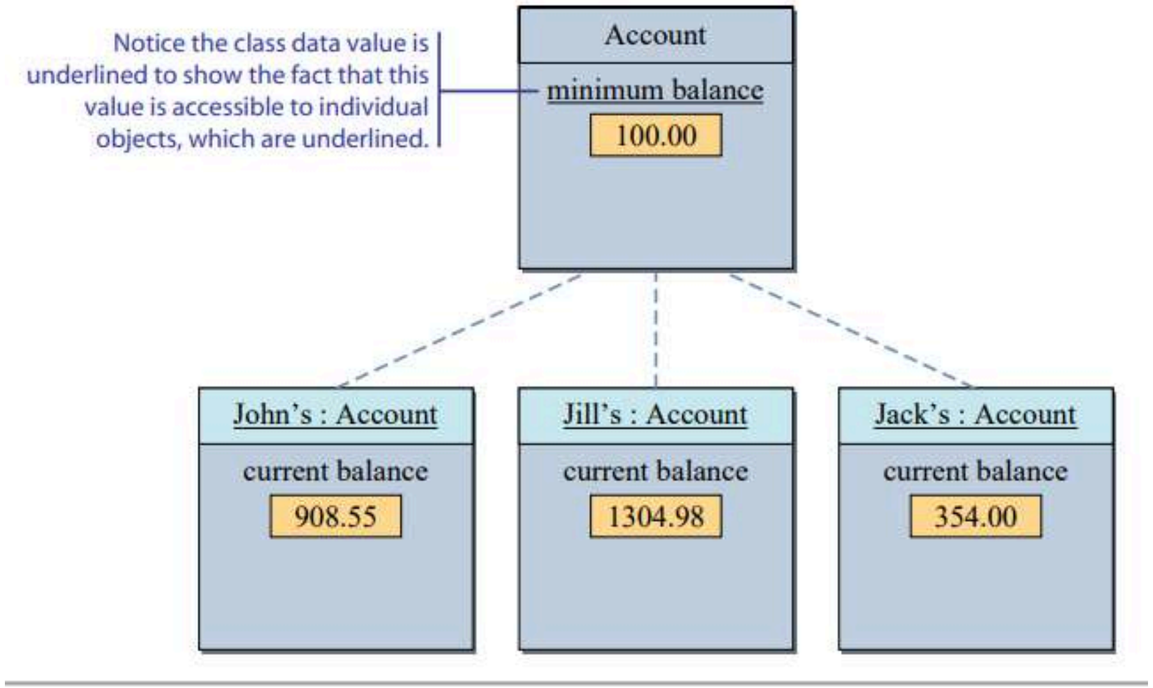


Figure 1.6 Three Account objects sharing information (minimum balance $100) stored as a class data value.

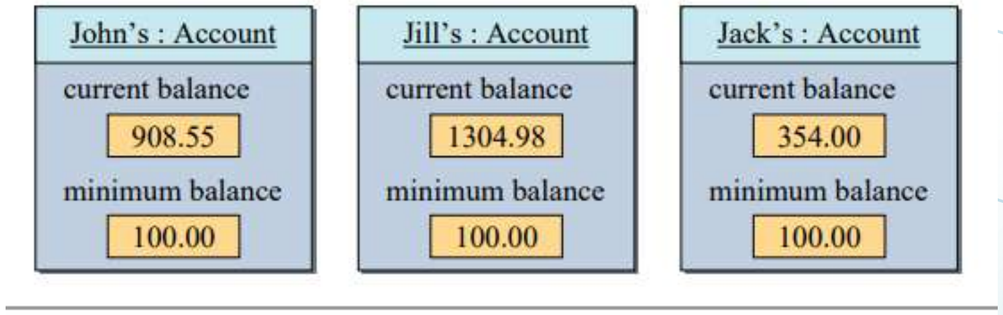


Figure 1.7 Three Account objects duplicating information (minimum balance $100) in instance data values.

Class and Instance Data Values

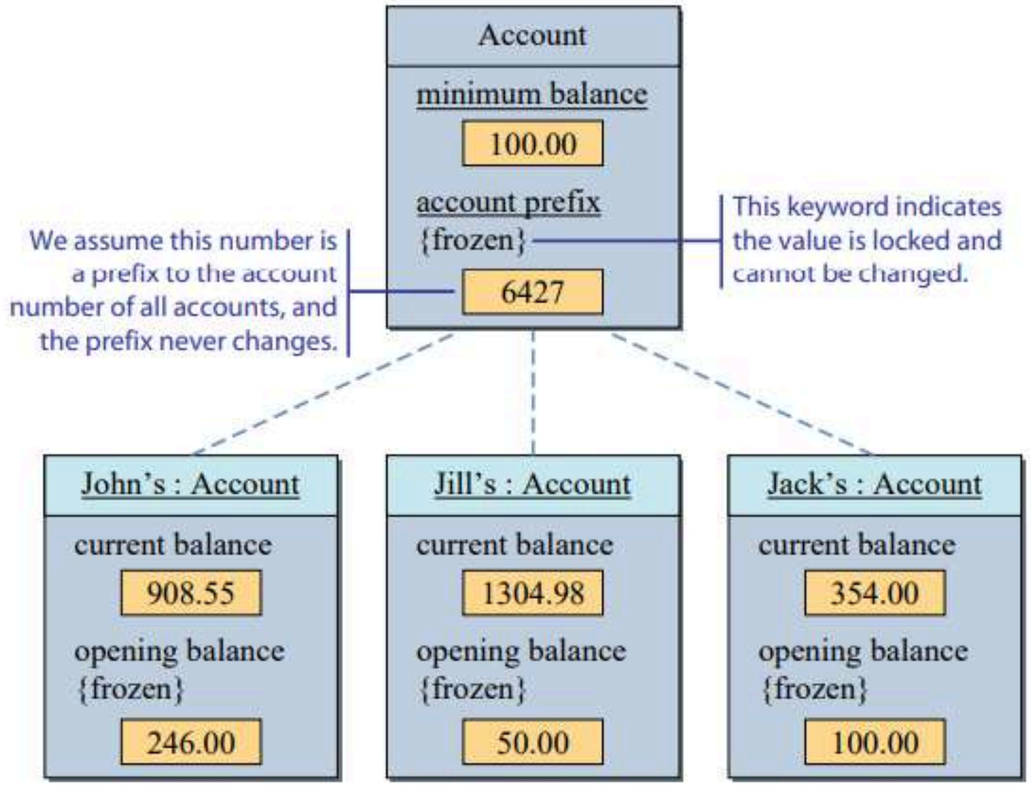


Figure 1.7 Graphical representations for four types of data values:
: class variable, class constant, instance variable, and instance constant.

- Analogous to defining class and instance methods, we can define **class and instance data values**.
- **A class data value** is used to represent information shared by all instances or to represent collective information about the instances.
- Data values are **also called data members** because they belong to a class or instance of the class.
- A data value that can change is called **a variable**, and one that cannot change is **a constant.**

Messages and Methods

In writing object-oriented programs we must first define classes, and while the program is running, we use the classes and objects from these classes to accomplish tasks.

- To instruct a class or an object to perform a task, we send a **message** to it.
  - For example, we send a message deposit to an Account object to deposit $100.
- For a class or an object to process the message it receives, it must possess a matching **method**, which is a sequence of instructions that a class or an object follows to perform a task.
  - A method defined for a class is called *a class method*.
  - A method defined for an object is *an instance method*.

Let's look at an example of an **instance method** first:

Suppose a **method called walk** is defined for a **Robot object** and **instructs the robot to walk a designated distance**. With this method defined, we can send the message walk to a Robot object, along with the distance to be walked. **A value we pass to an object is called an argument of a message.**

Notice that the name of the message we send to an object or a class must be the same as the method's name.

Messages and Methods (E.g)

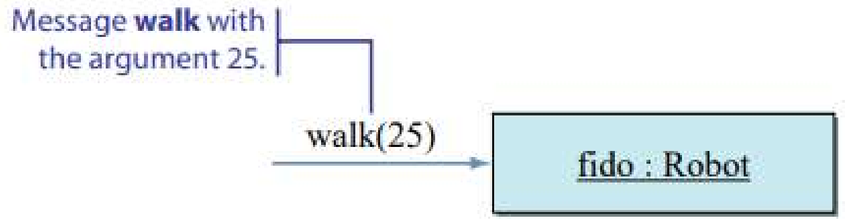


Figure 1.4 Sending the message walk to a Robot object.

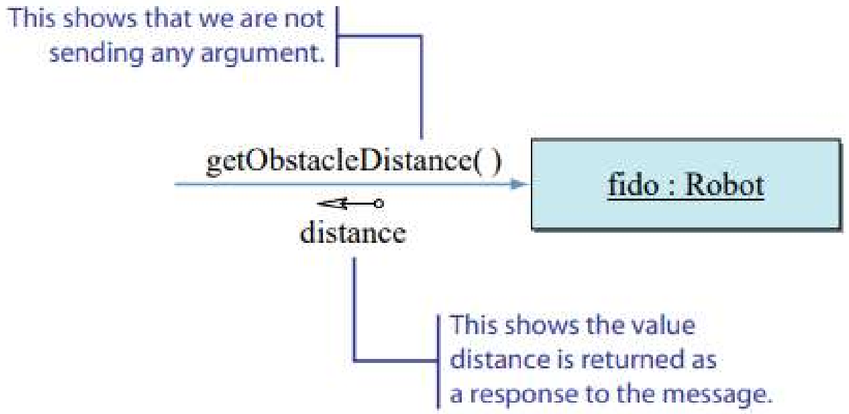


Figure 1.5 The result distance is returned to the sender of the message

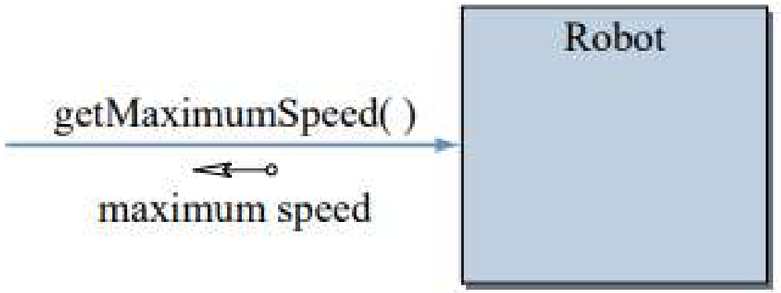


Figure 1.6 The maximum possible speed of all Robot objects is returned by the class method getMaximumSpeed.

Metode instans didefinisikan untuk tugas yang berkaitan dengan instans individual dan metode kelas untuk tugas yang berkaitan dengan semua instans.

Declare Class in Java

When you define a class, you declare its exact form and nature. **A class is declared by use of the *"class"* keyword.** In general, class declarations can include these components in order:

- *Modifiers*: A class can be public or have default access.
- *Class name*: The class name should begin with the initial letter capitalized by convention.
- *Body*: The class body is surrounded by braces, { }.

A simplified general form of a class definition is shown here:

```
public class classname {
      type instance-variable1;
      // ... type instance-variableN;

      type methodname1(parameter-list) {
          // body of method
      }
      // ...
      type methodnameN(parameter-list){
         // body of method
      }
}
```

Example Box:

```
public class Box {
   double width;
   double height;
   double depth;
}
```

Here is a class called **Box** that defines three instance variables: width, height, and depth. Currently, Box does not contain any methods *(but some will be added soon)*.

It is important to remember that **a class declaration only creates a template**; it does not create an actual object.

Declare Object in Java

Objects are the **instances of a class** that are created to use the **attributes** and **methods** of a class. A typical Java program creates many objects, which as you know, **interact by invoking methods**. When you create a class, you are creating a new data type. You can use this type to declare objects of that type.

An object consists of:

- *State*: It is represented by attributes of an object. It also reflects the properties of an object.
- *Behavior*: It is represented by the methods of an object. It also reflects the response of an object with other objects.
- *Identity*: It gives a unique name to an object and enables one object to interact with other objects.

There are three steps when creating an object from a class:

- **Declaration**: A variable declaration with a variable name with an object type.
- **Instantiation**: The 'new' keyword is used to create the object.
- **Initialization**: The 'new' keyword is followed by a call to a constructor. This call initializes the new object.

```
class-var = new classname();
```

```
Box mybox = new Box();
Box mybox; // declare reference to object
mybox = new Box(); // allocate a Box object
```

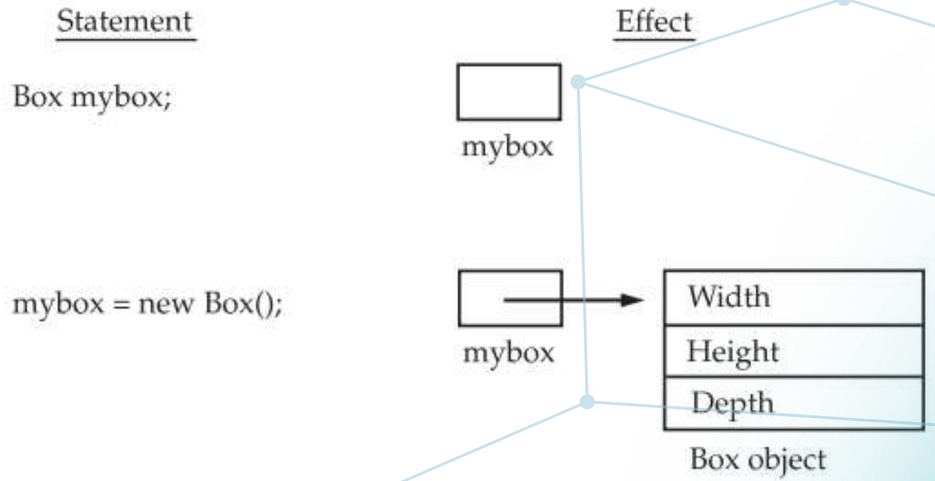


Figure 1.6 Declaring an object of type **Box**

Adding a Method

A class with only data fields has no life. Objects created by such a class cannot respond to any messages.

- Methods are declared inside the body of the class but immediately after the declaration of data fields.
- However, when an instance variable is accessed by code that is part of the same class as the instance variable, that variable can be referred to directly.

For example, adding a method to reffered by the class directly:

```
class classname {
  // ... type instance-variableN;

  type methodnameN(parameter-list){
    // body of method
  }
}
```

```
class Box {
   double width;
   double height;
   double depth;

   // display volume of a box
   void volume() {
      System.out.print("Volume is ");
      System.out.println(width * height * depth);
   }
}
```

Calling a Method

After we learn how to create our own methods with or without return values, then we will learn how to **invoke a method with or without parameters**, **overload methods using the same names**, and **apply method abstraction** in the program design. Methods are also known as Procedures or Functions:

- Procedures: They don't return any value.
- Functions: They return value.

For example, fere is the source code of the above defined method called **max()**. This method takes **two parameters** num1 and num2 and returns the maximum between the two:

```
public class ExampleMinNumber{
/** returns the minimum of two numbers */
   public static int minFunction(int n1, int n2) {
     int min;
     if (n1 > n2)
        min = n2;
     else
        min = n1;
     return min;
   }
   public static void main(String[] args) {
      int a = 11;
      int b = 6;

      /** how to call it */
      int c = minFunction(a, b);
      System.out.println("Minimum Value = " + c);
   }
}
```

Add and Call Method (E.g)

```
class BoxDemo3 {
   public static void main(String args[]) {
     Box mybox1 = new Box();
     Box mybox2 = new Box();

     // assign values to mybox1's instance variables
     mybox1.width = 10;
     mybox1.height = 20;
     mybox1.depth = 15;

     /* assign different values to mybox2's instance variables */
     mybox2.width = 3;
     mybox2.height = 6;
     mybox2.depth = 9;

     // display volume of first box
     mybox1.volume();
     // display volume of second box
     mybox2.volume();
   }
}
```

This program generates the following output, which is the same as the previous version.

```
Volume is 3000.0 Volume is 162.0
```

Look closely at the following two lines of code:

```
mybox1.volume();
mybox2.volume();
```

Notes:

- When an instance variable is accessed by code that is not part of the class in which that instance variable is defined, it must be done through an object, by use of the dot (.) operator.
- However, when an instance variable is accessed by code that is part of the same class as the instance variable, that variable can be referred to directly.

The Void Keyword

The void keyword allows us to create methods which do not return a value.

This method is **a void method** which does not return any value. Here, in the following example we're considering a void method **methodRankPoints.**

- Call to a void method must be a statement i.e. methodRankPoints(255.7);.
- It is a Java statement which ends with a semicolon as shown below

```
public class ExampleVoid {

   public static void main(String[] args) {
      methodRankPoints(255.7);
   }

   public static void methodRankPoints(double points) {
     if (points >= 202.5) {
       System.out.println("Rank:A1");
     } else if (points >= 122.4) {
       System.out.println("Rank:A2");
     } else {
       System.out.println("Rank:A3");
     }
   }
}
```

This would produce the following result:

```
Rank:A1
```

Passing Parameters by Value

While working under calling process, arguments is to be passed. These should be in the same order as their respective parameters in the method specification.

Parameters can be passed by value:

- **Passing Parameters by Value means calling a method with a parameter.**
- Through this the argument value is passed to the parameter.

The following program shows an example of passing parameter by value.

```
public class swappingExample {
  public static void swapFunction(int a, int b) {
    System.out.println("Before swapping(Inside),
    a = " +  a + " b = " + b);
    // Swap n1 with n2
    int c = a; a = b; b = c;
    System.out.println("After swapping(Inside), a = "
    + a + " b = " + b);
  }
  public static void main(String[] args) {
    int a = 30;
    int b = 45;
    System.out.println("Before swapping, a = "
    + a + " and b = " + b);
    // Invoke the swap method

    swapFunction(a, b);
    System.out.println("\n**Now, Before and After
    swapping values will be same here**:");
    System.out.println("After swapping, a = "
    + a + " and b is " + b);
  }
}
```

This would produce the following result: ???

Constructors of a Class

One of the most important subtopic would be *constructors*. **Every class has a constructor**. When an object of a class is created, the constructor is called automatically to initialize the object's attributes.

- If we do not explicitly write a constructor for a class the Java compiler builds a *default constructor* for that class.
  - **A default constructor** is a constructor that accepts no arguments and has no statements in its body.
- **No return type:** constructors do not have any return type, not even void.
- **Same name as the class:** a constructor has the same name as the class in which it is defined.
- A class can have more than one constructor.

For example, if we omit a constructor from the Bicycle class:

```
class Account{
......
  // A Constructor
  public Account() {

  }
.......
}
```

Once we define our own constructor, no default constructor is added. This means that once the constructor, such as :

```
public Account(String name, double startingBalance)
{
    ownerName = name;
    balance = startingBalance;
}
```

is added to the Account class, we will no longer be able to create a Account object anymore by executing

```
Account acct;
acct = new Account( );
```

because no matching constructor can be found in the class.

The [this] Keyword

Sometimes a method will need to refer to the object that invoked it. To allow this, Java defines the this keyword. Keyword **“this” can be used inside any method to refer to the current object.**

In the previous example if we change :

```
// A redundant use of this.
Box(double w, double h, double d) {
  width = w;
  height = h;
  depth = d;
}
```

To

```
Box(double width, double height, double depth) {
  width = width;
  height = height;
  depth = depth ;
}
```

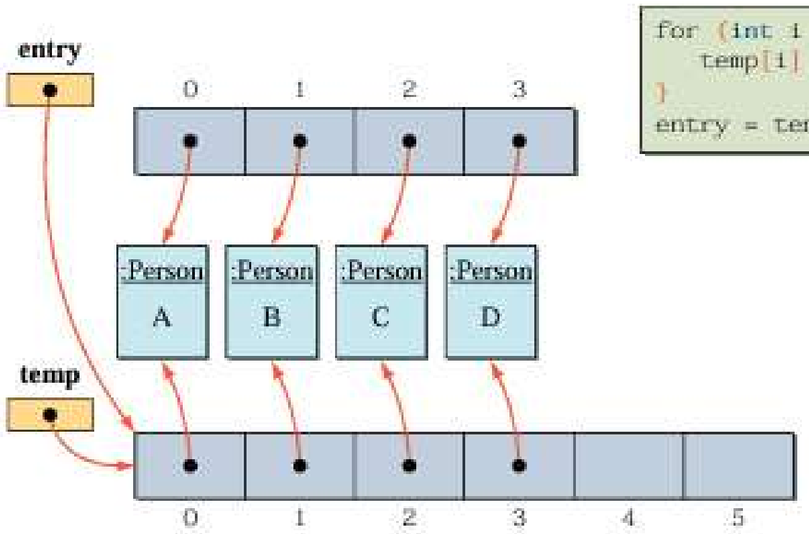
To resolve :

```
Box(double width, double height, double depth) {
  this.width = width;
  this.height = height;
  this.depth = depth ;
}
```

Garbage Collection

When an object has no references pointing to it, then the system will erase the object and make the memory space available for other uses. We call the erasing of an object deallocation of memory, and the process of deallocating memory is **called garbage collection**.

- If in the previous example we say
  `mybox2 = null;`
- The object mybox2 does not have a reference and cannot be used in future.
- The object becomes a candidate for automatic **garbage collection.**
- Java automatically collects garbage periodically and releases the memory used to be used in the future.




```
for (int i = 0; i < entry.length; i++) {
   temp[i] = entry[i];
}
entry = temp;
```

Note: The old array will eventually get returned to the system via garbage collection.

Quick Excercise: (1)

Let’s define another class for practice.

We learned in a chemistry class that an element is the simplest type of matter that consists of exactly one kind of atom. These elements have unique physical and chemical properties. The elements are classified into groups and periods in the periodic table. Each element is represented by its

1. name (e. g., hydrogen, helium, lithium, etc.);
2. atomic number (e.g., 1, 2, 3, etc.);
3. atomic symbol (e.g., H, He, Li, etc.);
4. atomic mass (e.g., 1.008, 4.003, 6.941, etc.);
5. period (it ranges from 1 to 7);
6. group (it ranges from 1 to 18 under the new system).

We name the class **Element**. The class **has one constructor** and **six accessors (get methods)** for the **six attributes**. We **do not have any mutators (set methods)** for this class because once an Element object is created we will never change the values of its attributes. Because all of its attributes must be specified, the constructor has six parameters.

Here’s the Element class: ???

Quick Excercise: (2)

```
class Element {

    //Data Members
    private String name;
    private int    number;
    private String symbol;
    private double mass;
    private int    period;
    private int    group;

    //Constructor
    public Element (String elementName, int elementNumber,
                    String elementSymbol, double elementMass,
                    int elementPeriod, int elementGroup) {

        name   = elementName;
        number = elementNumber;
        symbol = elementSymbol;
        mass   = elementMass;
        period = elementPeriod;
        group  = elementGroup;
    }

    //Returns the element's name
    public String getName () {
        return name;
    }

    //Returns the element's atomic number
    public int getNumber () {
        return number;
    }

    //Returns the element's 1- or 2-letter symbol
    public String getSymbol () {
        return symbol;
    }

    //Returns the element's atomic mass
    public double getMass () {
        return mass;
    }

    //Returns the element's period
    public int getPeriod () {
        return period;
    }

    //Returns the element's group
    public int getGroup () {
        return group;
    }
}
```

The following sample code creates three Element objects:

```
Element e1, e2, e3;
e1 = new Element ("Hydrogen", 1, "H", 1.008, 1, 1);
e2 = new Element ("Gold", 79, "Au", 197.0, 6, 11);
e3 = new Element ("Oxygen", 8, "O", 16.0, 2, 16);
```

Tips & Hint: Constructor

It is possible to define more than one constructor to a class. **Multiple contructors are called overloaded constructors.**

It is almost always a good idea to define multiple constructors to a class.

Always define **a constructor** and initialize data members fully in the constructor so an **object will be created in a valid state.**

Once a programmer has added an **explicitly defined constructor to a class**, no default constructor will be added to the class by the compiler

TERIMAKASIH

Stay Happy & Healthy